

How Large Language Models Work

A plain-language explainer covering tokenization, embeddings, the transformer architecture, training, text generation, and the limitations of modern LLMs.

1. Introduction: What Is a Large Language Model?

A large language model, or LLM, is a type of artificial intelligence system trained to predict and generate human language. At its core, an LLM is a statistical model that learns the patterns of language from enormous amounts of text. The word "large" refers to the number of parameters the model contains, which can range from hundreds of millions to hundreds of billions or more. Parameters are the adjustable numerical values inside the model that get tuned during training.

Despite their sophistication, LLMs do one fundamental thing: given a sequence of text, they predict what text is most likely to come next. Everything an LLM appears to do, including answering questions, writing code, translating languages, and summarizing documents, emerges from this single capability applied at massive scale. The model does not look up answers in a database. Instead, it generates responses one piece at a time based on statistical patterns learned during training.

2. Tokenization: Turning Text into Numbers

Computers cannot process raw text directly, so the first step is tokenization. Tokenization is the process of breaking text into smaller units called tokens. A token may be a whole word, part of a word, a single character, or a punctuation mark. For example, the word "unbelievable" might be split into the tokens "un", "believ", and "able". On average, one token corresponds to roughly four characters of English text, or about three-quarters of a word.

Each unique token is assigned an integer identifier from the model's vocabulary, which is a fixed list of all tokens the model knows. Modern vocabularies typically contain tens of thousands to over a hundred thousand tokens. The choice of tokenization scheme affects how efficiently the model handles different languages and rare words. Sub-word tokenization, used by most current models, strikes a balance between character-level and word-level approaches, allowing the model to handle words it has never seen by composing them from familiar pieces.

3. Embeddings: Giving Tokens Meaning

Once text has been split into tokens and converted to integer IDs, each token is mapped to a vector, which is a list of numbers. This vector is called an embedding. An embedding represents the meaning of a token as a point in a high-dimensional space, often with hundreds or thousands of dimensions. Tokens with similar meanings end up close together in this space. For instance, the embeddings for "king" and "queen" will be near each other, while "king" and "banana" will be far apart.

Embeddings are learned during training rather than assigned by hand. This is what allows a model to capture subtle relationships between words, such as analogies and categories. Importantly, the same embedding technique powers retrieval systems like the one in a

retrieval-augmented generation pipeline: documents and questions are converted into embeddings so that semantically similar pieces of text can be found by measuring the distance between their vectors.

4. The Transformer Architecture

Nearly all modern LLMs are built on an architecture called the transformer, introduced in 2017. The transformer was a breakthrough because it could process all tokens in a sequence in parallel rather than one at a time, making training far more efficient on modern hardware. A transformer is composed of a stack of identical layers, each of which refines the model's internal representation of the text.

Each layer contains two main components: a self-attention mechanism and a feed-forward network. The self-attention mechanism lets each token gather information from other tokens in the sequence, while the feed-forward network processes that information independently for each position. Information flows upward through the stack, with each layer building a richer understanding of context than the one below it.

5. Self-Attention: The Key Mechanism

Self-attention is the heart of the transformer and the reason LLMs handle context so well. When the model processes a token, self-attention allows it to weigh the importance of every other token in the input and focus on the ones most relevant to the current one. Consider the sentence "The trophy did not fit in the suitcase because it was too big." To understand what "it" refers to, the model must attend to "trophy" rather than "suitcase". Self-attention is what makes this possible.

Mechanically, each token produces three vectors: a query, a key, and a value. The model compares a token's query against the keys of all other tokens to compute attention scores, which determine how much each token contributes. These scores are used to produce a weighted blend of the value vectors. Models use many attention heads in parallel, each able to focus on different kinds of relationships, such as grammar, subject matter, or position. This is called multi-head attention.

6. Layers, Depth, and Parameters

The capability of an LLM is closely tied to its scale. Stacking more layers gives the model more opportunities to combine and abstract information, while wider layers let it represent more detail at each step. The total count of all the weights across the embeddings, attention mechanisms, and feed-forward networks is what people mean when they refer to a model's parameter count. A model described as having seventy billion parameters has roughly that many individual numbers that were tuned during training.

In general, larger models trained on more data tend to be more capable, but scale is not the only factor. The quality of the training data, the training method, and the way the model is fine-tuned all have a major effect on real-world performance.

7. Training Part One: Pretraining

Training an LLM happens in stages. The first and most computationally expensive stage is pretraining. During pretraining, the model is shown vast quantities of text from books, websites, articles, and code, and is given one simple task: predict the next token. The model makes a prediction, compares it to the actual next token, and adjusts its parameters slightly to make a better prediction next time. This adjustment is performed using an algorithm called backpropagation combined with gradient descent.

Repeated across trillions of tokens, this process gradually shapes the model's parameters so that it captures grammar, facts, reasoning patterns, and writing styles. Pretraining is self-supervised, meaning it does not require humans to label the data; the text itself provides the correct answer, since the next token is always known. The result of pretraining is a base model that is good at continuing text but not yet tuned to follow instructions or hold a helpful conversation.

8. Training Part Two: Fine-Tuning and Instruction Tuning

A raw base model will happily continue any text, but it does not reliably do what a user asks. Fine-tuning addresses this. In fine-tuning, the pretrained model is trained further on a smaller, more carefully curated dataset. Instruction tuning is a common form of fine-tuning in which the model learns from many examples of instructions paired with high-quality responses. This teaches the model the format and behavior of being a helpful assistant: answering questions directly, following formatting requests, and staying on topic.

Fine-tuning is far cheaper than pretraining because it uses much less data and far fewer computing resources. It is also how a general model can be specialized for a particular domain, such as medicine or law, by training it on domain-specific examples.

9. Training Part Three: Alignment with Human Feedback

The final common stage is alignment, often done through a technique called reinforcement learning from human feedback, or RLHF. The goal of alignment is to make the model's outputs more helpful, honest, and harmless according to human preferences. In RLHF, humans compare different responses the model produces and indicate which they prefer. These preferences are used to train a separate reward model that scores responses.

The language model is then optimized to produce responses that the reward model rates highly. This process nudges the model toward answers people actually find useful and away from outputs that are unsafe, evasive, or low quality. Alignment is a major reason modern assistants feel polite, cooperative, and reluctant to produce harmful content.

10. Inference: How Text Is Actually Generated

Once trained, using the model to produce output is called inference. LLMs generate text autoregressively, which means one token at a time, from left to right. The model takes the input prompt, predicts a probability distribution over all possible next tokens, selects one, appends it to the sequence, and then repeats the entire process using the newly extended sequence as the new input.

This loop continues until the model produces a special stop token or reaches a length limit. Because each new token depends on all the tokens before it, the model can stay coherent across long passages. It also means generation is inherently sequential, which is why producing a long response takes longer than a short one.

11. Sampling Strategies and Temperature

At each step the model outputs probabilities, but it still must choose a single token. The strategy for making this choice is called sampling, and it strongly affects the character of the output. The simplest method, greedy decoding, always picks the single most likely token. This is predictable but can be repetitive and dull.

A parameter called temperature controls how random the selection is. A low temperature, near zero, makes the model almost always pick the most probable token, producing focused and deterministic output that is well suited to factual tasks. A high temperature flattens the probabilities, making less likely tokens more competitive and producing more creative or varied output. Two related techniques, top-k sampling and top-p sampling, restrict the choice to a limited set of the most probable tokens, which helps avoid nonsensical words while still allowing variety.

12. The Context Window

An LLM can only consider a limited amount of text at once. This limit is called the context window, and it is measured in tokens. The context window includes both the input prompt and the response being generated. If a conversation or document exceeds the window, the oldest content must be dropped or summarized, and the model effectively forgets it.

Context window sizes have grown dramatically, from a few thousand tokens in early models to hundreds of thousands or even millions in recent ones. A larger window lets the model work with longer documents and remember more of a conversation, but processing very long contexts requires more computation and memory. The context window is also why retrieval-augmented generation is useful: instead of feeding an entire library into the model, a retrieval system finds only the most relevant passages and places them in the context window alongside the question.

13. Emergent Capabilities

One of the most striking features of large models is that certain abilities appear only once a model reaches sufficient scale. These are called emergent capabilities because they are not present in smaller models and were not explicitly programmed. Examples include multi-step reasoning, performing arithmetic, translating between languages the model was not specifically trained to translate, and following complex instructions given as examples within the prompt.

A related technique is in-context learning, where the model adapts its behavior based on examples provided in the prompt itself, without any change to its parameters. Showing the model a few examples of a task before asking it to perform that task is known as few-shot prompting, and it often dramatically improves results.

14. Limitations and Risks

LLMs are powerful but imperfect, and understanding their limitations is essential. The most well-known is hallucination, where the model produces confident statements that are factually wrong. This happens because the model generates statistically plausible text rather than retrieving verified facts; it has no built-in sense of truth and cannot tell when it is wrong.

A second limitation is the knowledge cutoff. A model only knows what was in its training data, so it is unaware of events that occurred after training ended. Connecting the model to live search or a retrieval system is a common way to address this. Other risks include bias inherited from training data, sensitivity to small changes in how a prompt is worded, and difficulty with precise calculations or tasks requiring exact recall. These limitations are why grounding a model in trusted source documents, as a retrieval-augmented system does, can make its answers more reliable and verifiable.

15. Glossary of Key Terms

Token: A small unit of text, such as a word or part of a word, that the model processes.

Embedding: A vector of numbers representing the meaning of a token or piece of text.

Parameter: An adjustable numerical value inside the model, tuned during training.

Transformer: The neural network architecture underlying modern LLMs.

Self-attention: The mechanism that lets each token draw on other tokens for context.

Pretraining: The initial stage of training on large text using next-token prediction.

Fine-tuning: Additional training on curated data to specialize behavior.

RLHF: Reinforcement learning from human feedback, used to align model outputs with human preferences.

Inference: The process of using a trained model to generate output.

Temperature: A setting that controls the randomness of generated text.

Context window: The maximum amount of text, in tokens, a model can consider at once.

Hallucination: A confident but factually incorrect output from the model.